Randomized Ski Rental and RandomizedSkiQF

Background and analysis

November 2026

Abstract

This report separates classical ski-rental theory from its application to RandomizedSkiQF. Part I explains the classical rental model, the randomized purchase rule, and its expected competitive guarantee. Part II develops workload-dependent cost comparisons relevant to RandomizedSkiQF, including uniform occurrence counts and query-frequency distributions. The latter calculations are consequences of the idealized rental model, not measurements or a competitive proof for the integrated filter.

Part I

The classical randomized ski-rental algorithm

1 The decision problem

You need skis on each day of a season, but do not know how many days the season will last. Renting costs r per day. Buying costs br once and covers that day and all later days. Divide all costs by r: renting now costs 1 and buying costs b. Buying replaces renting on the purchase day; it does not incur an additional day's rental charge.

Initially assume that $b = B$ is a positive integer. The season length $n \geq 1$ is an integer revealed one day at a time. On day k, you know that day k exists, but not whether day $k+1$ will exist. The price is fixed, purchase is always available, and ownership is permanent. These assumptions define the problem being analyzed.

An offline algorithm knows n in advance. It either rents throughout or buys on the first day, so

$$\mathrm{OPT}(n) = \min(n, B).$$

An online algorithm has to decide without that knowledge. If it plans to buy on day K, its total cost is

$$C(n, K) = \begin{cases} n, & n < K, \\ K - 1 + B, & n \geq K. \end{cases}$$

The purchase happens only if the season reaches day K.

2 The deterministic baseline

The standard deterministic rule rents for $B - 1$ days and buys on day B. If $n < B$, it is optimal. If $n \geq B$, its cost is $2B - 1$, compared with offline cost B. Its worst-case ratio is therefore

$$\rho_{\mathrm{det}} = 2 - \frac{1}{B}.$$

For integer B, this is optimal among deterministic algorithms. Buying earlier on day $k < B$ lets the adversary stop on day k, giving ratio $(k-1+B)/k = 1+(B-1)/k \geq 2-1/B$. Buying later gives a long-season cost greater than $2B-1$; never buying has unbounded ratio. Before purchasing, the only new information is that the season continues, so a deterministic policy is characterized by such a purchase day.

The vulnerability is uncertainty about the stopping time. A purchase just before the season ends wastes money, while waiting too long wastes rental payments. Randomization balances these risks across different season lengths.

3 What "competitive in expectation" means

For a randomized algorithm, K is random. A ratio ρ means

$$\text{for every fixed } n \geq 1, \qquad \mathbb{E}_K[C(n,K)] \leq \rho\,\mathrm{OPT}(n).$$

Equivalently, the competitive ratio is $\sup_n \mathbb{E}_K[C(n,K)]/\mathrm{OPT}(n)$. The expectation is over the algorithm's private random choices. No probability distribution on season lengths is assumed.

This is the oblivious-adversary model: the adversary can know the purchase distribution but must choose the season independently of the realized random choices. An adversary that observes the purchase and immediately ends the season changes the problem. Also, the inequality is an expected-cost statement, not a guarantee for every run or a high-probability bound.

The classical optimal limiting ratio is $e/(e-1) \simeq 1.582$, as recalled in the ski-rental literature [2]. The frequently quoted deterministic ratio 2 and randomized ratio 1.582 are limiting values; integer purchase costs admit the sharper finite-B expressions below.

4 The randomized purchase rule

For $B \geq 2$, define $q = 1 - 1/B$ and sample

$$\boxed{\Pr(K=k) = p_k = \frac{q^{B-k}}{B(1-q^B)}, \qquad k = 1,\ldots,B.}$$

Then rent on days before K and buy on day K, if it occurs. This is not a uniform random threshold: later days have larger probability, with $p_{k+1} = p_k/q$. The geometric-series identity gives

$$\sum_{k=1}^{B} p_k = \frac{1-q^B}{B(1-q)(1-q^B)} = 1.$$

For $B = 1$, buy on the first day deterministically.

The algorithm makes one random choice per season, before observing its length:

1. Sample one purchase day K from $p_1,\ldots,p_B$.
2. Rent on every day before K that actually occurs.
3. If day K occurs, buy then; otherwise the season ends without a purchase.

Once K has been sampled, all subsequent actions are deterministic. There is no further random decision during the season.

To sample this distribution concretely, draw one uniform number $U \in [0,1)$ and choose the smallest c such that $U < \sum_{j=1}^{c} p_j$. The interval assigned to day c has length p_c, so the probability of selecting that day is exactly p_c.

5 Deriving the purchase-day distribution

Karlin et al. describe a randomized algorithm as a probability distribution over deterministic threshold algorithms. They write the inequalities needed for a competitive guarantee, set them to equalities, solve the resulting difference equations, and normalize the probabilities [1]. The following gives that construction in ski-rental notation; the original paper uses a caching problem with different cost conventions.

Let p_c be the probability of choosing the deterministic algorithm that buys on day c. For a season of length $n \le B$, its expected cost is

$$E_n = \sum_{c=1}^{n} (B + c - 1)p_c + n \sum_{c=n+1}^{B} p_c.$$

The first sum covers choices that buy before the season ends. The second covers choices that rent throughout. For $n \ge B$, every choice has bought, so $E_n = E_B$.

Our objective is to choose nonnegative probabilities summing to one and the smallest possible ρ such that

$$\begin{cases} E_n \le \rho n, & 1 \le n \le B, \\ E_n \le \rho B, & n > B. \end{cases}$$

These are simply the competitive-ratio requirements, since the offline cost is $\min(n, B)$. Together with $p_c \ge 0$ and $\sum_c p_c = 1$, minimizing ρ is a linear program: each expected cost E_n is a linear function of the probabilities. Only the first B cost constraints are needed because costs stop changing after day B.

Geometrically, we lower ρ until lowering it further would make the constraints infeasible. An optimum can be chosen at a vertex of the feasible polyhedron, where some constraints are tight. Linear programming alone does not imply that every cost constraint is tight. For this ski-rental construction, setting all B cost constraints to equality produces the classical candidate distribution.

Following Karlin's construction, set these inequalities to equalities and solve for the probabilities. For the first B horizons, this means

$$E_1 = \rho, \quad E_2 = 2\rho, \quad \ldots, \quad E_B = B\rho.$$

This step looks for a distribution that balances the expected ratio across season lengths. It is a way to construct the algorithm, not an extra assumption about the input or the definition of competitiveness. After solving, we verify that the probabilities are valid and that they satisfy the required bound. Proving that no other algorithm does better is a separate lower-bound argument.

Step 1: Express the cost of one more day. Couple seasons of length $c - 1$ and c by using the same planned purchase day K. If $K < c$, both seasons have already paid for skis and the cost increment is zero. If $K = c$, the longer season buys for B, whereas the shorter season stops after renting for $c - 1$ days. If $K > c$, the longer season pays one more rental. Therefore

$$E_c - E_{c-1} = Bp_c + \sum_{k=c+1}^{B} p_k. \tag{1}$$

Since we set $E_c = \rho c$ and $E_{c-1} = \rho(c - 1)$, subtracting gives $E_c - E_{c-1} = \rho$. For $c = 1$, use $E_0 = 0$.

Step 2: Eliminate the unknown ratio. For consecutive days c and $c+1$, with $c < B$, we require

$$Bp_c + p_{c+1} + \sum_{k=c+2}^{B} p_k = \rho,$$
$$Bp_{c+1} + \sum_{k=c+2}^{B} p_k = \rho.$$

Subtracting the second equation from the first gives

$$Bp_c - (B-1)p_{c+1} = 0, \qquad p_c = \left(1 - \frac{1}{B}\right) p_{c+1} = qp_{c+1}.$$

Iterating backward from the last day yields $p_c = q^{B-c} p_B$.

Step 3: Normalize the distribution. Since the policy buys on one of days $1, \ldots, B$,

$$1 = \sum_{c=1}^{B} p_c = p_B \sum_{j=0}^{B-1} q^j = p_B \frac{1-q^B}{1-q}.$$

Thus $p_B = (1-q)/(1-q^B)$, and substitution gives

$$\boxed{p_c = \frac{(1-q)q^{B-c}}{1-q^B} = \frac{q^{B-c}}{B(1-q^B)}.}$$

All terms are positive for $B > 1$, so this is a valid probability distribution. On day B, equation (1) gives $\rho = Bp_B = 1/(1-q^B)$. The recurrence ensures every earlier increment has that same value. Thus the equations produce a valid distribution with the desired expected costs.

6 Deriving the expected guarantee

The recurrence and normalization above give

$$E_c - E_{c-1} = \rho_B, \qquad \rho_B = \frac{1}{1-(1-1/B)^B}, \quad 1 \le c \le B.$$

Starting from $E_0 = 0$ and summing these increments gives $E_n = \rho_B n$ for $n \le B$. For $n > B$, every possible sampled purchase day has already occurred, so $E_n = E_B = \rho_B B$. Combining the two cases,

$$\boxed{\mathbb{E}_K[C(n,K)] = \rho_B \min(n,B) = \rho_B \mathrm{OPT}(n).}$$

This proves the expected guarantee for every fixed season length. The separate assertion that no randomized policy can achieve a smaller worst-case ratio requires a minimax lower-bound proof, which is not reproduced here.

Since $(1-1/B)^B \to e^{-1}$,

$$\rho_B \longrightarrow \frac{e}{e-1} = 1.5819767\ldots.$$

For finite integer $B > 1$, ρ_B is smaller than this limit and smaller than $2 - 1/B$.

Purchase cost B	Randomized ratio	Deterministic ratio
1	1.0000	1.0000
2	1.3333	1.5000
3	1.4211	1.6667
5	1.4874	1.8000
10	1.5353	1.9000
$B \to \infty$	1.5820	2.0000

7 A better worst-case guarantee does not mean lower cost on every season

For $n < B$, deterministic ski rental pays n, whereas the randomized rule pays $\rho_B n$ in expectation. Randomization pays a premium on short seasons. For $n \geq B$, it saves

$$\Delta_B = 2B - 1 - \rho_B B$$

relative to the deterministic rule. It trades some short-season performance for better protection when the season is long.

If a separate application model supplies a random horizon N, independent of the algorithm's coins, the randomized rule has lower average modeled cost precisely when

$$\Pr(N \geq B)\Delta_B > (\rho_B - 1)\mathbb{E}[N\mathbf{1}_{\{N<B\}}].$$

This is an algebraic comparison of the two cost functions, not an assumption needed by competitive analysis.

Part II
Workload implications for RandomizedSkiQF

8 The application model and its scope

The following calculations apply the classical cost functions to idealized independent adaptation instances. Let a missed opportunity to adapt incur a false-positive database lookup of cost R, and let adaptation add cost A. Under the convention that the triggering lookup must also be paid, the normalized purchase cost is $B = 1 + A/R$. We assume this is an integer and that both policies use this calibrated cost. An instance with N occurrences would incur N false positives if it were never corrected; a successful purchase eliminates the remaining ones.

For general $\alpha = A/R$, with deterministic threshold t, the total cost is k when $k < t$ and $t + \alpha$ otherwise. For a sampled threshold with probabilities p_c,

$$E[C(k, K)] = k \Pr(K > k) + \sum_{c=1}^{\min(k,B)} p_c(c + \alpha).$$

The classical simplification $\rho_B \min(k, B)$ requires integer $B = 1+\alpha$. For any other supplied threshold and cost ratio, use the full expression rather than claiming that same calibrated guarantee.

These assumptions let us compare policies mathematically before measuring the system. They do not identify every query key with an independent instance in the actual filter: fingerprints can share state, and one adaptation can affect several keys. Space limits, failed adaptations, CPU work, and changing access costs also fall outside the formulas. No implementation changes are made or assumed here.

9 Known workload distributions change the optimization objective

Competitive analysis minimizes a worst-case ratio over horizons. If instead the horizon N follows a known distribution, an average-cost objective is

$$\min_{\text{policy}} \mathbb{E}_{N,K}[C(N, K)].$$

These objectives can prefer different policies. In particular, the classical randomized rule is not designed to minimize expected cost for every distribution of N.

For a fixed purchase threshold k, define

$$L(k) = \mathbb{E}_N[C(N, k)] = \sum_{n=0}^{k-1} n \Pr(N = n) + (k - 1 + B) \Pr(N \geq k).$$

Allow $k = \infty$ to mean never buy, with $L(\infty) = \mathbb{E}[N]$. A randomized threshold, sampled independently of N with probabilities a_k, has average cost

$$\mathbb{E}_{N,K}[C(N, K)] = \sum_k a_k L(k) \geq \inf_k L(k).$$

For finite workloads the minimum is attained, so a best deterministic threshold achieves at least as low an average cost as any mixture. This is a linear objective over a probability simplex: select a threshold with smallest coefficient $L(k)$. It does not say that the standard deterministic threshold $k = B$ is best for every known workload distribution. Nor does it address policies that observe additional predictive information beyond the occurrence count.

10 Which occurrence distributions favor randomization?

For comparison with deterministic purchase at B, define

$$\Delta_B = 2B - 1 - \rho_B B.$$

For $B > 1$, this is positive. On each instance shorter than B, the randomized policy pays an expected premium $(\rho_B - 1)N$. On each instance reaching B, it saves Δ_B. Averaging gives

$$\mathbb{E}[C_{\text{rand}} - C_{\text{det}}] = (\rho_B - 1)\mathbb{E}[N\mathbf{1}_{\{N<B\}}] - \Delta_B \Pr(N \geq B). \tag{2}$$

Thus the sign depends on both the probability of long instances and the total expected rental cost of short instances. If every instance has $N < B$, deterministic rental is offline-optimal on each one. If every instance has $N \geq B$, randomization has strictly smaller expected cost for $B > 1$. A distribution being "random" by itself implies neither conclusion.

10.1 The selected once-or-twice experiment

Let f denote the fraction of distinct absent query keys occurring once. Every other key occurs twice. At nominal $\alpha = 1$, $B = 2$, the predicted costs per initially false-positive instance are

$$E[C_{\text{det}}] = 3 - 2f, \qquad E[C_{\text{rand}}] = (8 - 4f)/3.$$

The ideal crossover is $f = 1/2$, with randomization favored below it. These are modeled costs, not predictions that physical I/O or throughput must cross at the same point.

11 Uniform query keys versus uniform occurrence counts

Uniformly sampling query keys is different from uniformly sampling each key's total occurrence count. Suppose T query keys are drawn independently, and key i has probability w_i. Its total count has marginal distribution

$$N_i \sim \text{Binomial}(T, w_i).$$

The counts across keys need not be independent for their expected costs to sum. Under the idealized per-key adaptation model, apply equation (2) to each relevant key and add the results. Keys with zero occurrences contribute zero cost.

For uniform queries over M keys, $w_i = 1/M$. The parameter T/M controls the expected count per key. If it is small relative to the purchase threshold, short instances may dominate and early purchases can be wasteful. With a sufficiently long workload over the same finite key space, counts reach B with high probability and the comparison can reverse. Therefore uniform queries do not, by themselves, guarantee that deterministic SkiQF is better.

For a normal or lognormal query generator, the same calculation applies once its discrete key probabilities w_i are specified. If key i corresponds to an interval $[a_i, b_i)$ under a cumulative distribution function F, its probability is $F(b_i) - F(a_i)$, with renormalization when the generator truncates the support. Substituting these probabilities into the binomial count distribution gives

$$\Pr(N_i \geq B) = \sum_{n=B}^{T} \binom{T}{n} w_i^n (1 - w_i)^{T-n},$$

$$\mathbb{E}[N_i \mathbf{1}_{\{N_i<B\}}] = \sum_{n=0}^{\min(B-1,T)} n \binom{T}{n} w_i^n (1 - w_i)^{T-n}.$$

A sum with lower limit exceeding its upper limit is zero. These expressions determine the expected policy difference without a separate competitive proof for each named distribution. The actual generator's rounding, truncation, domain size, and parameters matter; "normal" or "lognormal" alone is insufficient.

These count distributions describe potential occurrences before adaptation, not the observed count of false positives after a policy starts correcting them. Applying the calculation to RandomizedSkiQF requires a justified mapping from those potential occurrences to the fingerprint-level instances. That mapping and a full filter-wide guarantee remain separate work.

12 Execution decisions and experiment status

The selected families are distributions (uniform, normal, lognormal, Zipf), the existing immediate-feedback adaptive adversary, and singleton mixtures with $f = 0, 0.25, 0.50, 0.75, 1$. All five filters participate. Each case has one repetition at seed 741262640; these are single-run measurements.

Medium runs use $q = 24$, ten million queries per static case, and thirty million queries for the adaptive adversary. Large runs use $q = 27$ and one hundred million queries per case. The existing medium and large adversary detector windows remain 100,000 and 1,000,000 negative queries, respectively. All runs retain eight remainder bits, 90% initial occupancy, 100 reporting rounds, nominal $B = 2$, and the existing storage budgets. Large runs must not begin before all medium cases and their reports pass validation.

The new workload uses ordinary absent keys rather than forced false positives. A seeded bijection over disjoint integer ranges produces unique inserted and query keys. For exactly T operations, set $D = \text{round}(T(1 - f)/(2 - f))$ doubleton keys and $S = T - 2D$ singleton keys, then shuffle the multiset. Report the realized $S/(S + D)$ as well as the requested fraction. All filters receive the same complete trace, including queries already corrected by a filter.

Workload histograms are computed outside timing after the trace is saved. Report singleton fractions both among distinct keys and among query operations. The nonadaptive baseline supplies a separate initial-false-positive frequency histogram; adaptive-filter frequencies are censored by corrections. Actual normal/lognormal generator behavior is preserved, and measured histograms characterize those traces.

The improved classical randomized cost guarantee requires an oblivious adversary. It is not a stronger guarantee for an adaptive filter against a non-oblivious adversary. No claim that this policy is automatically 2-competitive against that stronger adversary is made here. The policy implementation remains unchanged.

Execute from the benchmark worktree using:

```
.venv/bin/python experiments/run.py run --tests distributions \
  adaptive-adversary singleton-mixture --size medium --repetitions 1 \
  --output sponge/tods/randomized-medium
```

The lines above form one shell command. The staged entry point `.venv/bin/python experiments/run_randomized_study.py` enforces the medium-completion gate, updates this document with generated plots, and then executes large. Its execution log is `/tmp/randomized-study-execution.log`. After medium completion and report validation, use the same selection with `-size large` and a fresh `randomized-large` output directory. No other experiment families are selected.

13 Measured results

Medium results

All ten cases completed. Each measurement is from one run.

distributions/uniform: singleton keys 100.00%, singleton operations 100.00%; initial-FP singleton keys 100.00%; randomized/deterministic throughput 0.851.

distributions/normal: singleton keys 4.54%, singleton operations 0.90%; initial-FP singleton keys 4.98%; randomized/deterministic throughput 1.349.

distributions/lognormal: singleton keys 32.62%, singleton operations 9.42%; initial-FP singleton keys 32.01%; randomized/deterministic throughput 0.867.

distributions/zipf: singleton keys 70.17%, singleton operations 14.27%; initial-FP singleton keys 68.53%; randomized/deterministic throughput 0.931.

singleton-mixture/f-000: singleton keys 0.00%, singleton operations 0.00%; initial-FP singleton keys 0.00%; randomized/deterministic throughput 1.162.

singleton-mixture/f-025: singleton keys 25.00%, singleton operations 14.29%; initial-FP singleton keys 25.54%; randomized/deterministic throughput 1.057.

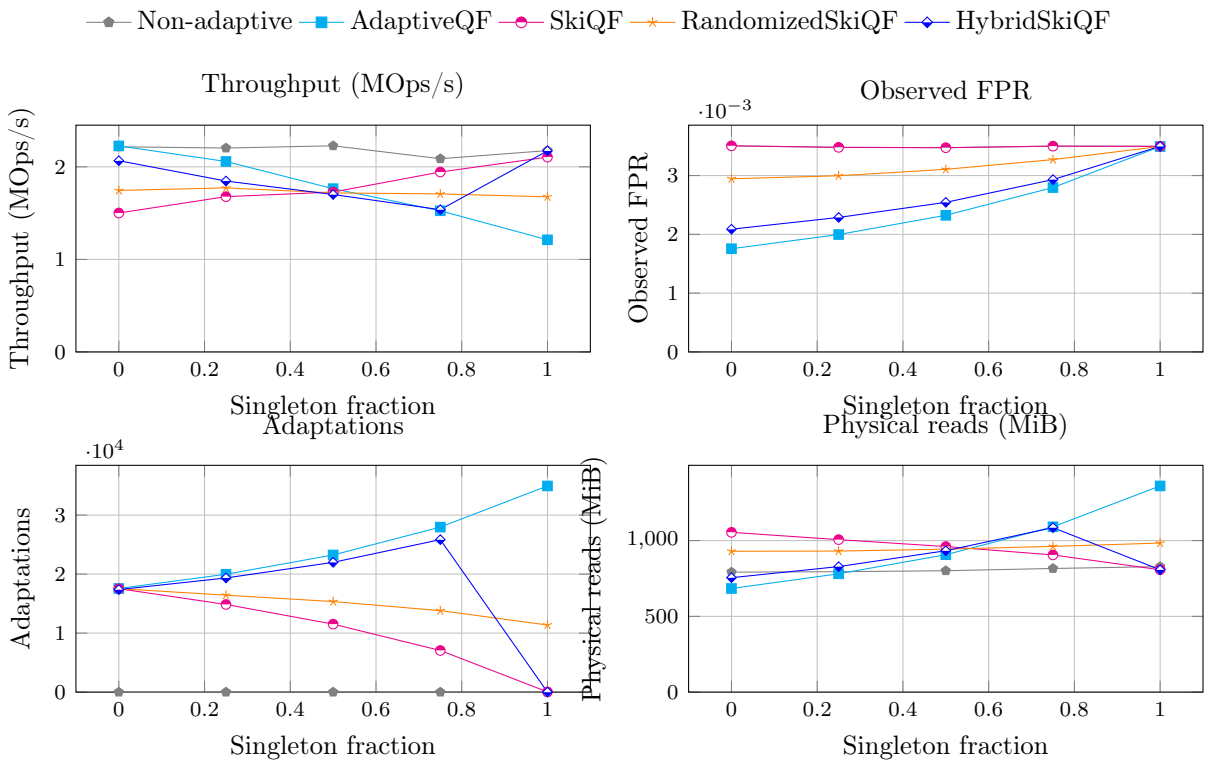
singleton-mixture/f-050: singleton keys 50.00%, singleton operations 33.33%; initial-FP singleton keys 50.41%; randomized/deterministic throughput 0.995.

singleton-mixture/f-075: singleton keys 75.00%, singleton operations 60.00%; initial-FP singleton keys 74.80%; randomized/deterministic throughput 0.878.

singleton-mixture/f-100: singleton keys 100.00%, singleton operations 100.00%; initial-FP singleton keys 100.00%; randomized/deterministic throughput 0.796.

14 Singleton mixtures

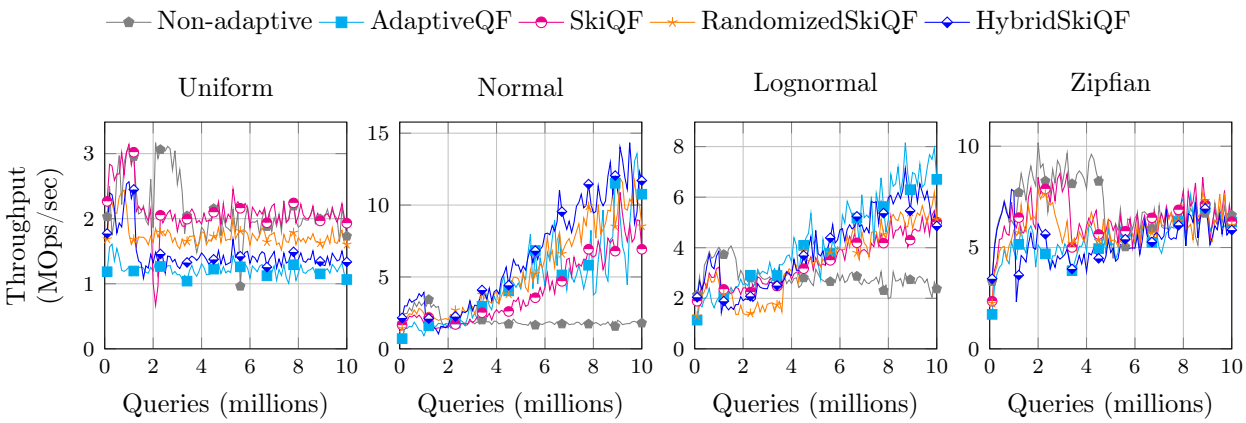
The horizontal axis is the fraction of distinct keys appearing once; all other keys appear twice.



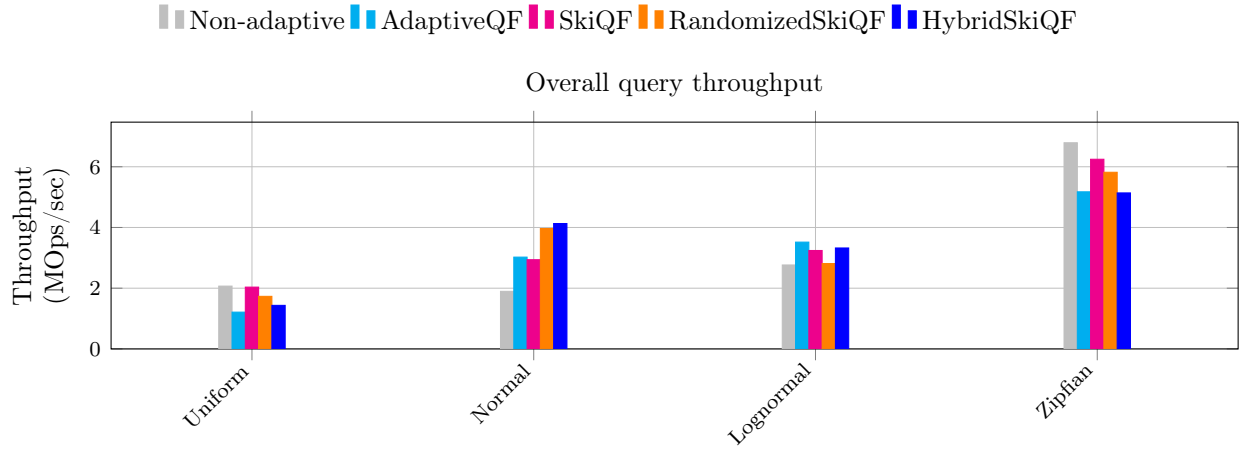
Single-run results; no error bars.

15 Robustness across workload distributions

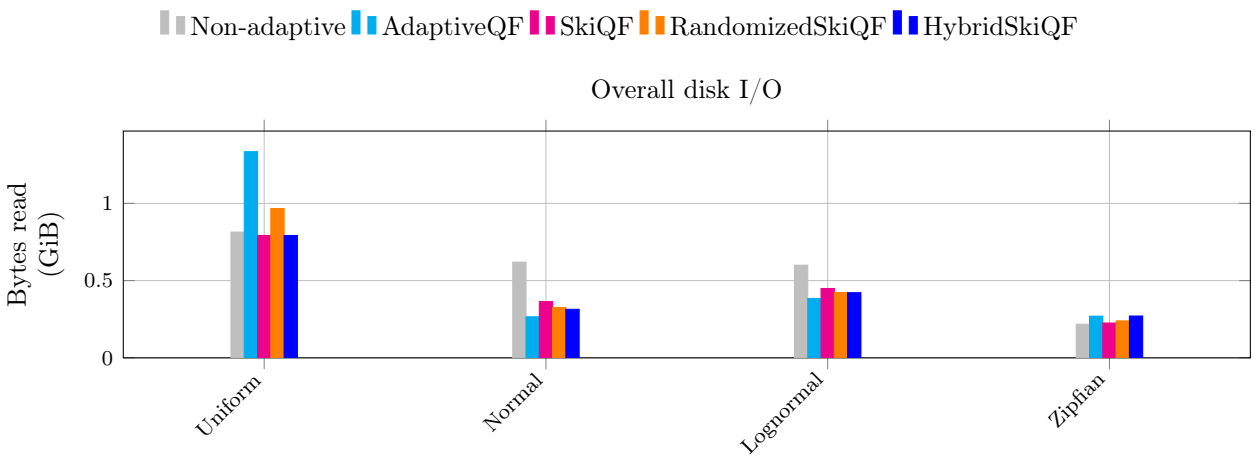
Distribution panels and aggregate comparisons follow the paper's robustness study.



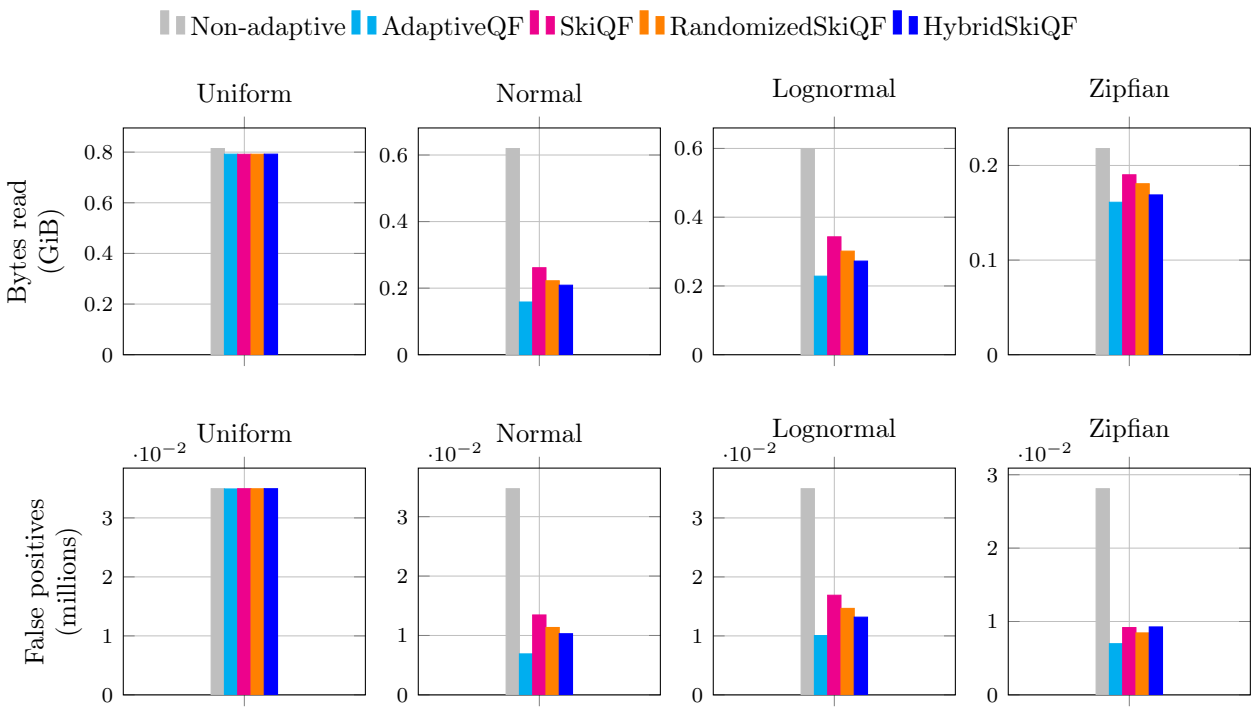
Instantaneous database query throughput. Higher is better.



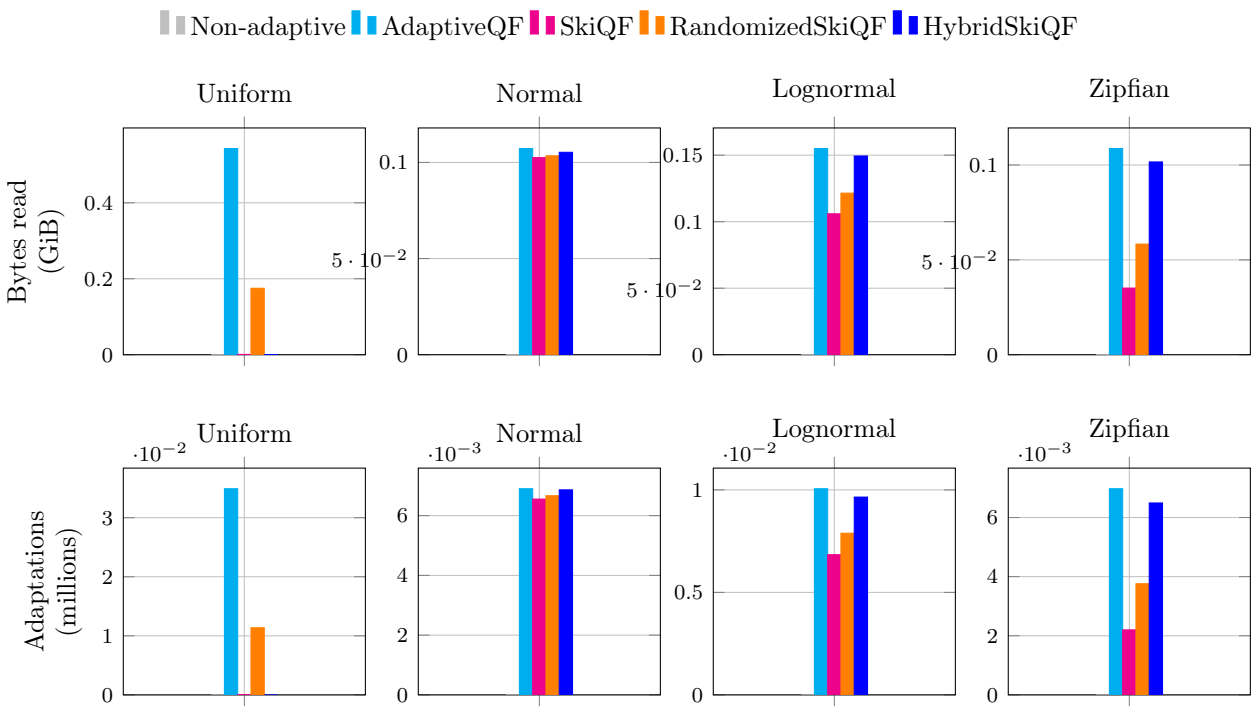
Final cumulative throughput; higher is better.



Database plus reverse-map physical reads. Lower is better.



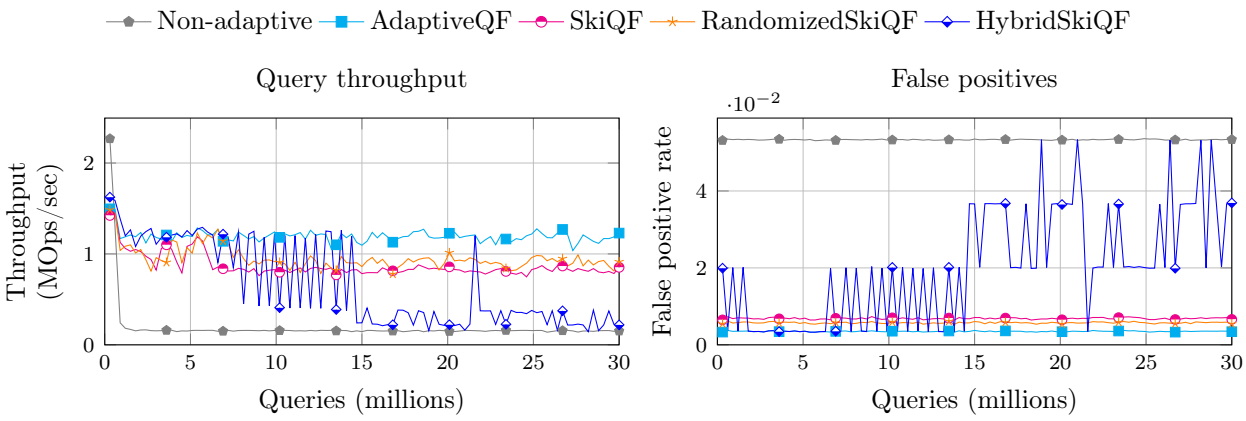
Database I/O and false positives. Lower is better.



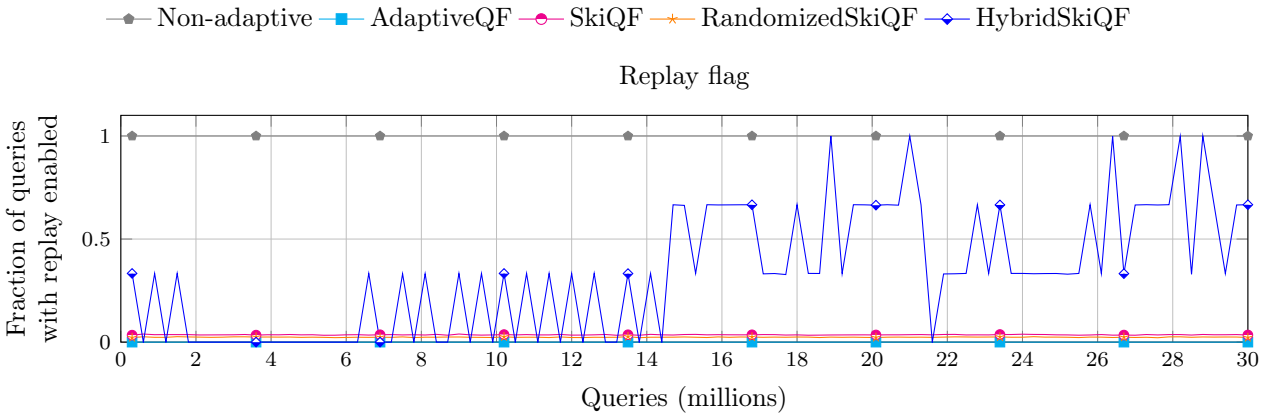
Reverse-map I/O and adaptations. Lower is better.

16 Adaptive adversary

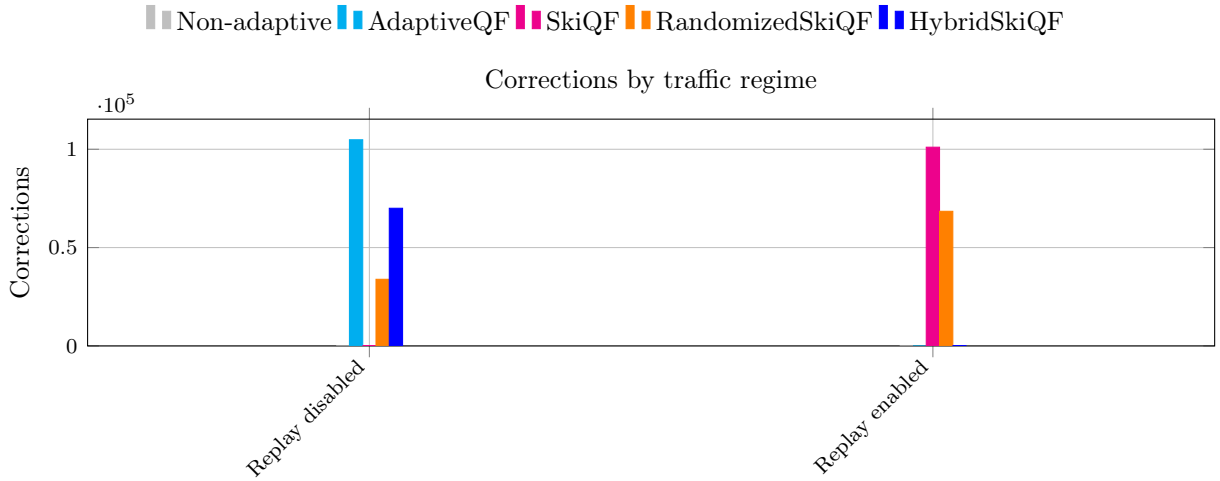
This new comparison extends the paper's adversarial study. Each filter receives its own feedback-dependent stream; each case identifies its feedback model.



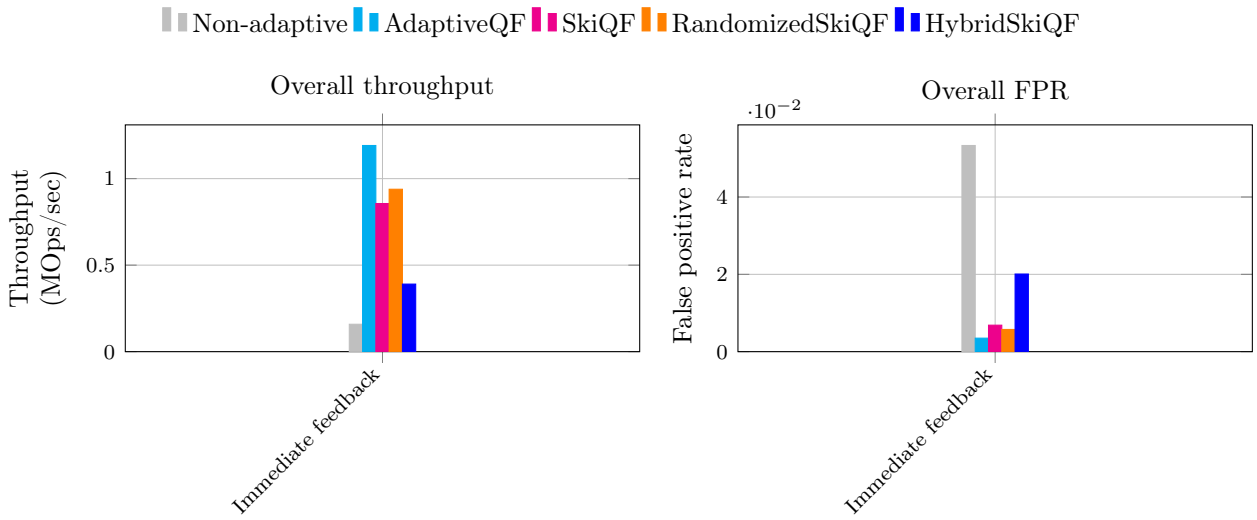
Perfect recognition of ordinary query outcomes: corrections immediately disable replay; tolerated false positives enable it. There are no probes or adversary windows. HybridSkiQF's detector window is 100,000 negative-key queries; smoothing remains 0.7 and the repeated-FP threshold remains 6.



The flag changes immediately; the plot averages its value over each reporting interval.



Regimes refer to the flag before the query; a correction disables replay for the next query.



Final cumulative measurements. Throughput: higher is better; FPR: lower is better.

Large results

All ten cases completed. Each measurement is from one run.

distributions/uniform: singleton keys 100.00%, singleton operations 100.00%; initial-FP singleton keys 100.00%; randomized/deterministic throughput 0.781.

distributions/normal: singleton keys 4.42%, singleton operations 0.88%; initial-FP singleton keys 4.52%; randomized/deterministic throughput 1.108.

distributions/lognormal: singleton keys 32.64%, singleton operations 9.42%; initial-FP singleton keys 32.59%; randomized/deterministic throughput 1.044.

distributions/zipf: singleton keys 71.09%, singleton operations 13.32%; initial-FP singleton keys 71.24%; randomized/deterministic throughput 0.932.

singleton-mixture/f-000: singleton keys 0.00%, singleton operations 0.00%; initial-FP singleton keys 0.00%; randomized/deterministic throughput 1.100.

singleton-mixture/f-025: singleton keys 25.00%, singleton operations 14.29%; initial-FP singleton keys 25.03%; randomized/deterministic throughput 1.052.

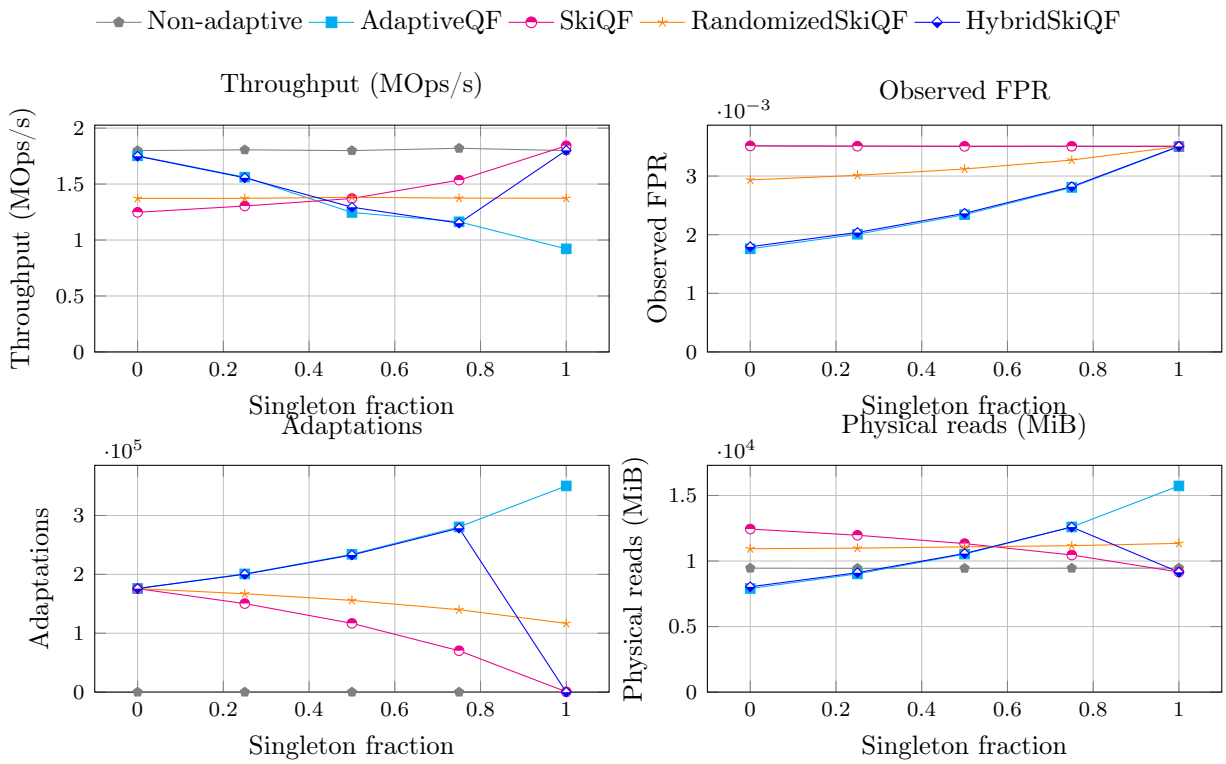
singleton-mixture/f-050: singleton keys 50.00%, singleton operations 33.33%; initial-FP singleton keys 50.11%; randomized/deterministic throughput 1.008.

singleton-mixture/f-075: singleton keys 75.00%, singleton operations 60.00%; initial-FP singleton keys 75.05%; randomized/deterministic throughput 0.895.

singleton-mixture/f-100: singleton keys 100.00%, singleton operations 100.00%; initial-FP singleton keys 100.00%; randomized/deterministic throughput 0.745.

17 Singleton mixtures

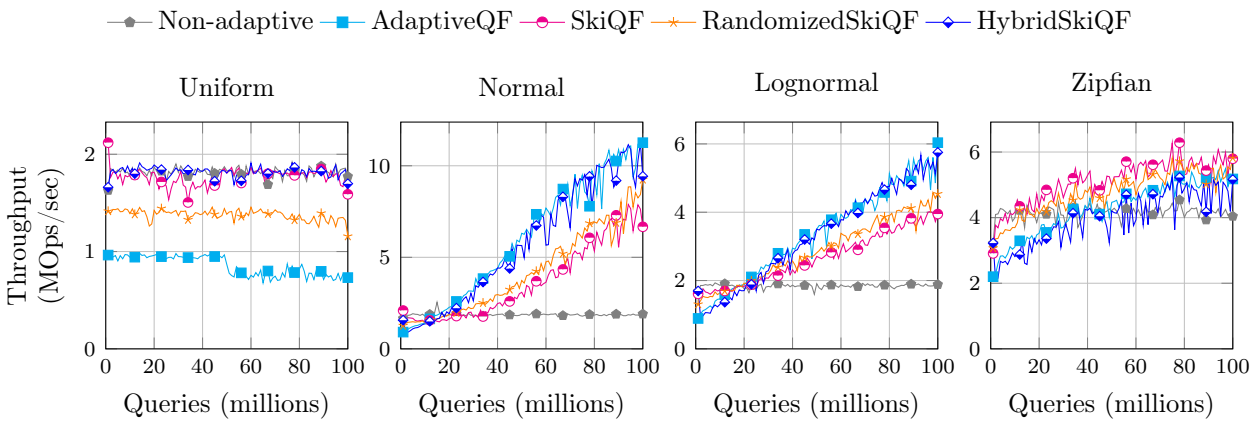
The horizontal axis is the fraction of distinct keys appearing once; all other keys appear twice.



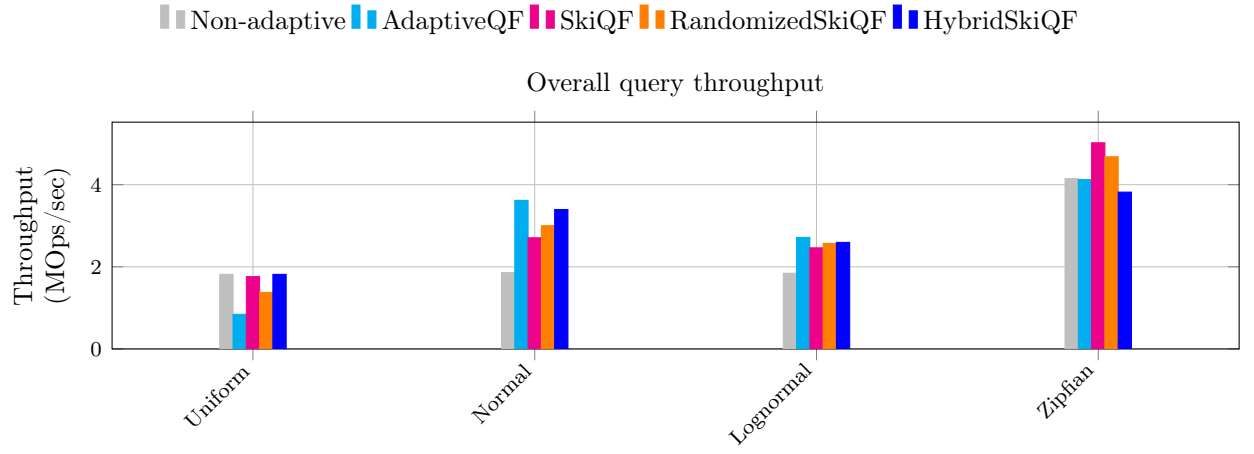
Single-run results; no error bars.

18 Robustness across workload distributions

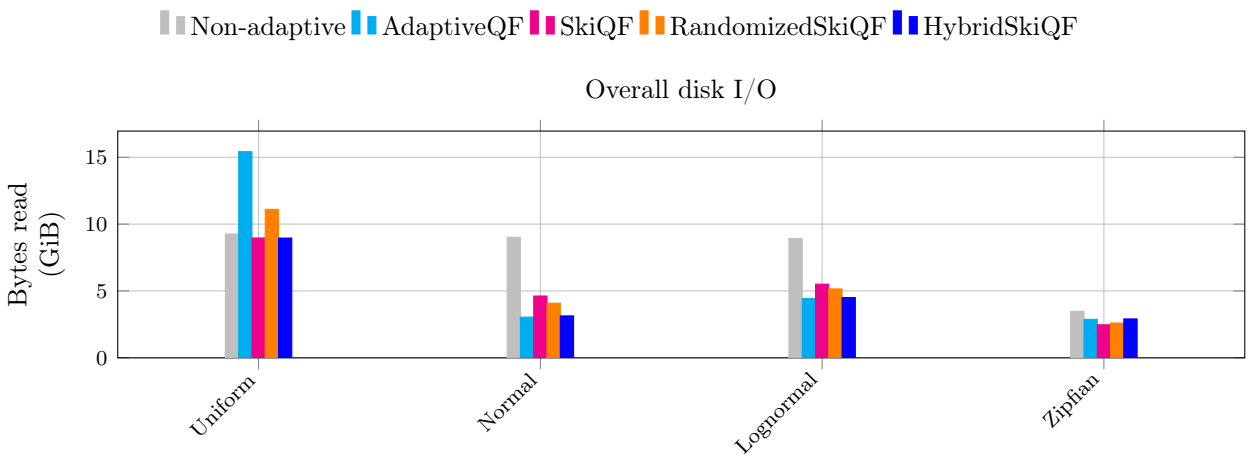
Distribution panels and aggregate comparisons follow the paper's robustness study.



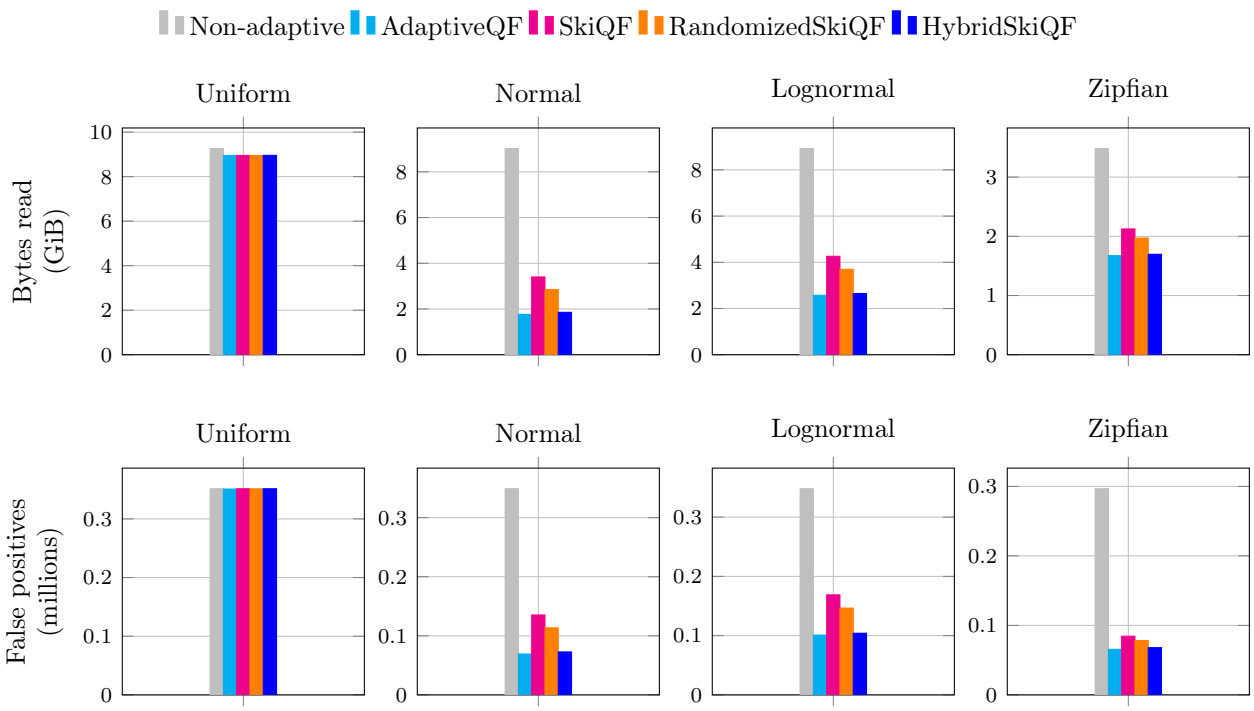
Instantaneous database query throughput. Higher is better.



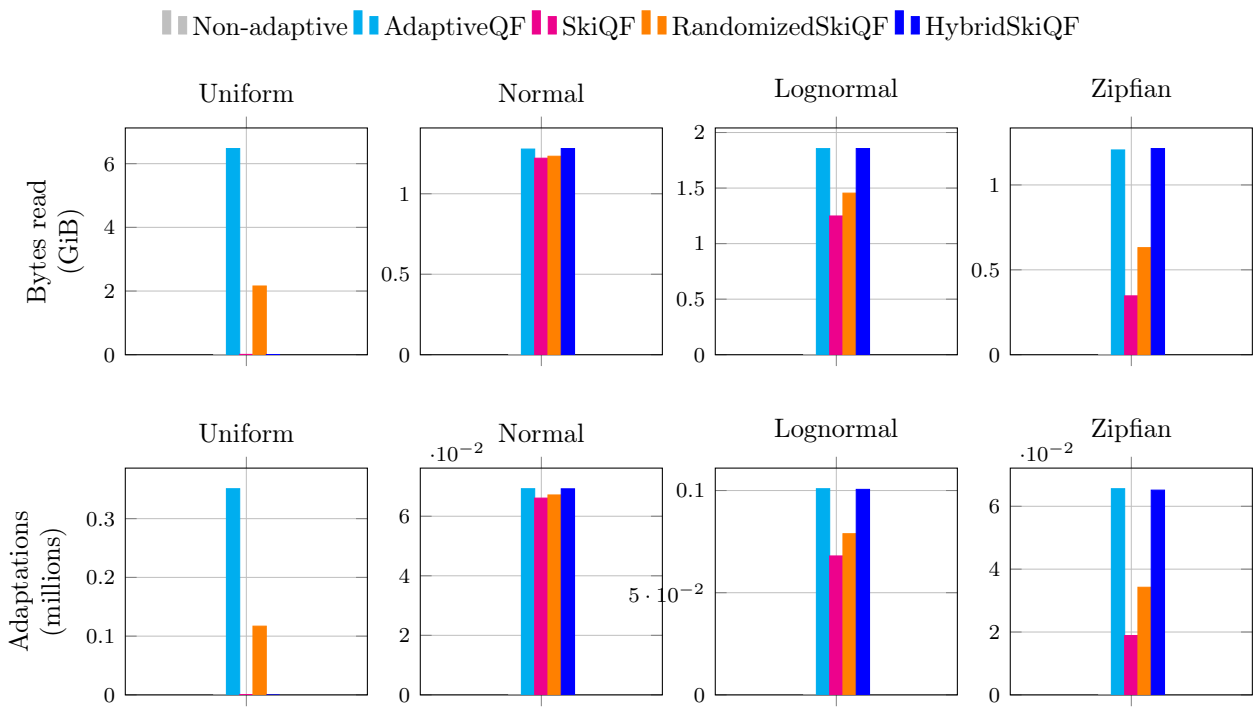
Final cumulative throughput; higher is better.



Database plus reverse-map physical reads. Lower is better.



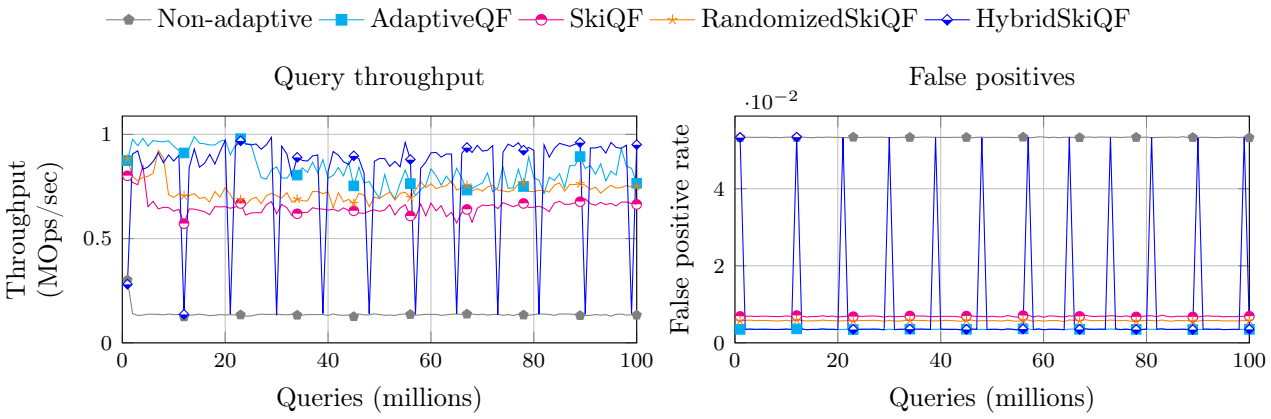
Database I/O and false positives. Lower is better.



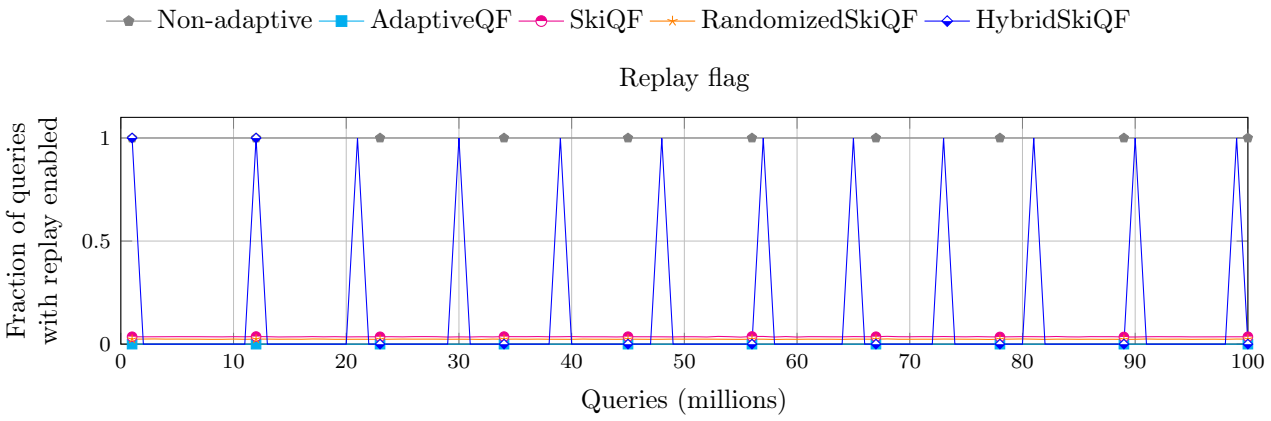
Reverse-map I/O and adaptations. Lower is better.

19 Adaptive adversary

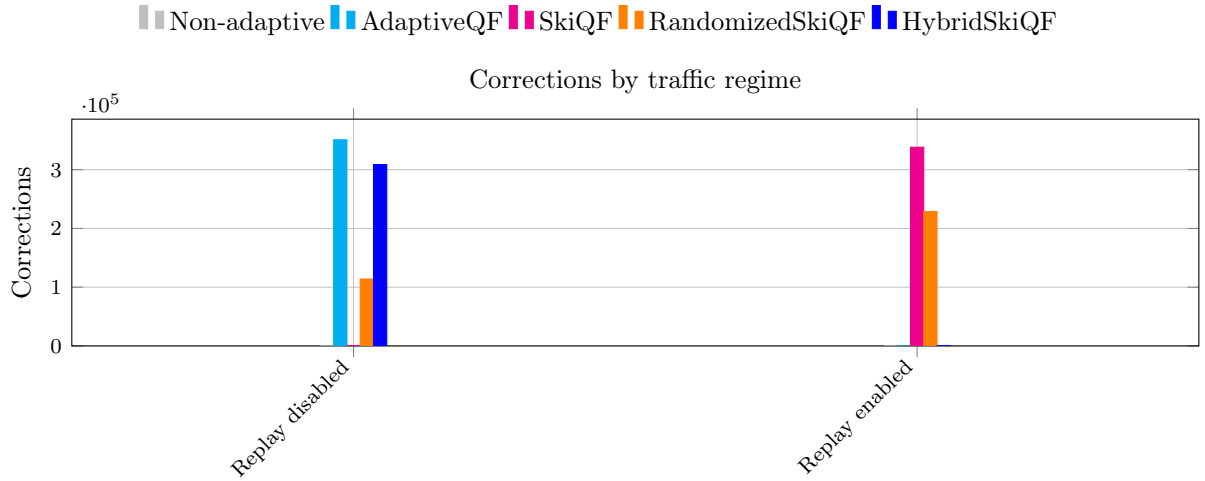
This new comparison extends the paper's adversarial study. Each filter receives its own feedback-dependent stream; each case identifies its feedback model.



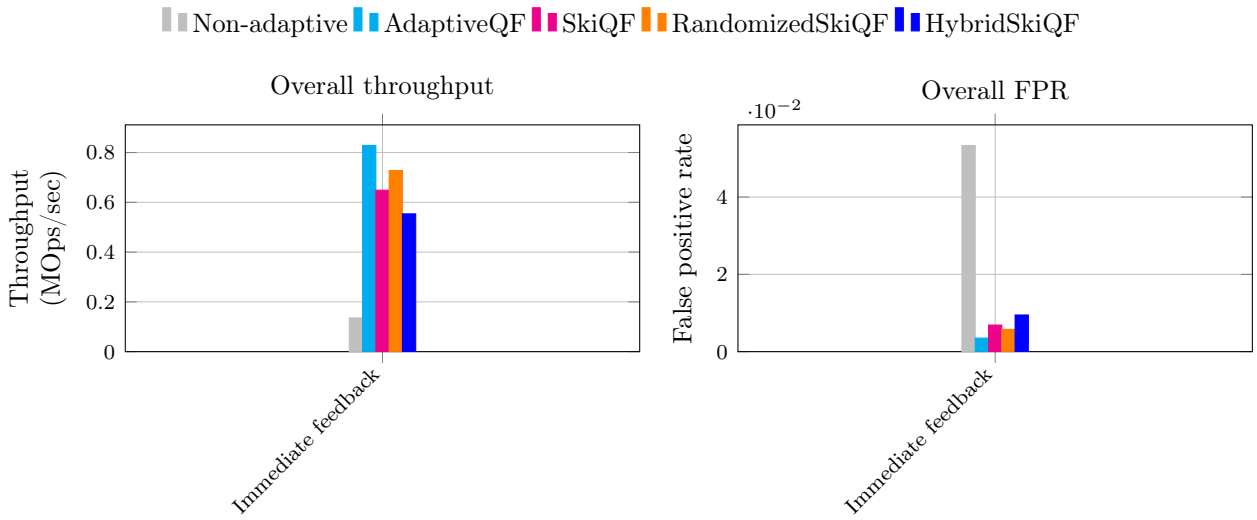
Perfect recognition of ordinary query outcomes: corrections immediately disable replay; tolerated false positives enable it. There are no probes or adversary windows. HybridSkiQF's detector window is 1,000,000 negative-key queries; smoothing remains 0.7 and the repeated-FP threshold remains 6.



The flag changes immediately; the plot averages its value over each reporting interval.



Regimes refer to the flag before the query; a correction disables replay for the next query.



Final cumulative measurements. Throughput: higher is better; FPR: lower is better.

References

[1] A. R. Karlin, M. S. Manasse, L. A. McGeoch, and S. Owicki, *Competitive Randomized Algorithms for Non-Uniform Problems*, SODA 1990, pp. 301–309, especially Theorem 1, p. 303. `https://courses.csail.mit.edu/6.895/fall03/handouts/papers/karlin.pdf`.

[2] A. Mądry and D. Panigrahi, *The Semi-stochastic Ski-rental Problem*, introduction and discussion of the classical model. `https://madry.mit.edu/docs/skirental.pdf`.